COL100/COL1000 Assignment 1

Submission Instructions

Write your solution in the provided `assignment.py` starter file. The function signatures are already defined. Do **not** add new functions or change the existing function names, parameters, or return types. Simply replace each `pass` statement with your implementation.

To test your solution against the visible test cases, place both `assignment.py` and `test_visible.py` in the **same folder** and run:

```
python test_visible.py
```

or, if your system requires it:

```
python3 test_visible.py
```

Note that passing all visible test cases does **not** guarantee full marks. Your code will also be evaluated against hidden test cases.

You can submit **only** the `assignment.py` file, exactly as named, without renaming or restructuring it.

Question 1: Digit Dominance

Write a Python function:

```
def digit_dominance(n: int) -> str:
```

The function takes a positive integer n. Using a `while` loop, count the number of even digits and odd digits in n.

The function should return:

- `"MoreEven"` if the number of even digits is greater than the number of odd digits.
- `"MoreOdd"` if the number of odd digits is greater than the number of even digits.
- `"Equal"` otherwise.

Note: The digit 0 should be counted as an even digit.

Test Cases

Test Case 1

Function call:

```
digit_dominance(9607004)
```

Expected return value:

```
"MoreEven"
```

Explanation: Even digits are $6, 0, 0, 4$, and odd digits are $9, 7$.

Test Case 2

Function call:

```
digit_dominance(13579)
```

Expected return value:

```
"MoreOdd"
```

Test Case 3

Function call:

```
digit_dominance(1234)
```

Expected return value:

```
"Equal"
```

Question 2: Number Hill Pattern

Write a Python function:

```
def number_hill(n: int) -> str:
```

The function takes a positive integer n and returns the number hill pattern as a multi-line string.

For example, if $n = 4$, the function should return:

```
1
121
12321
1234321
12321
121
1
```

The pattern first increases from row length 1 to row length $2n - 1$, and then decreases back to row length 1.

Use nested `for` loops.

Test Cases

Test Case 1

Function call:

```
number_hill(3)
```

Expected return value:

```
1
121
12321
121
1
```

Test Case 2

Function call:

```
number_hill(5)
```

Expected return value:

```
1
121
12321
1234321
123454321
1234321
12321
121
1
```

Question 3: First Repeated Digit

Write a Python function:

```
def first_repeated_digit(n: int) -> str:
```

The function takes a positive integer n. Starting from the rightmost digit, check whether any digit appears more than once.

The function should return the first digit in string format, from right to left, that has already appeared before.

If no digit is repeated, return:

`"No Repetition"`

Hint: You can use nested `while` loops. Use `break` once a repeated digit is found.

Test Cases

Test Case 1

Function call:

```
first_repeated_digit(527352)
```

Expected return value:

`"2"`

Explanation: Scanning from right to left, 2 appears again later in the number.

Test Case 2

Function call:

```
first_repeated_digit(987654)
```

Expected return value:

`"No Repetition"`

Test Case 3

Function call:

```
first_repeated_digit(10023)
```

Expected return value:

`"0"`

Test Case 4

Function call:

```
first_repeated_digit(4543214)
```

Expected return value:

`"4"`

Question 4: Hollow Right Triangle Pattern

Write a Python function:

```
def hollow_right_triangle(n: int) -> str:
```

The function takes a positive integer n and returns a hollow right-angled triangle of height n using `*`.

For example, if $n = 5$, the function should return:

```
*
**
* *
*  *
*****
```

Rules:

- The first row contains one `*`.
- The second row contains two `*`.
- The last row contains n stars.
- For rows between the second and last row, return `*`, then spaces, then `*`.

Use loops and string concatenation.

Test Cases

Test Case 1

Function call:

```
hollow_right_triangle(4)
```

Expected return value:

```
*
**
* *
****
```

Test Case 2

Function call:

```
hollow_right_triangle(6)
```

Expected return value:

```
*
**
* *
*   *
*     *
******
```